

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



DATA STRUCTURES AND ALGORITHMS (CO2003)

Extended assignment

Image deduplication

Instructor(s): TS. Lê Thành Sách

Class: TN01

Group: 2

Students: Vũ Hoàng Hải - 2410917
Nguyễn Hoàng Gia Huy - 2411222
Nguyễn Ngọc Thạch - 2413218

HO CHI MINH CITY, NOVEMBER 2025



SOURCE CODE VÀ DEMO

Toàn bộ mã nguồn và môi trường thực thi được cung cấp tại:

- **GitHub Repository:**
<https://github.com/Shurayz287/DSA-TN-Group02>
- **Google Colab Notebook:**
https://colab.research.google.com/drive/1qnNAR6L04_aJmjJ4-KL6pFaq2hTB2kfY



Contents

List of Figures	5
List of Tables	5
1 Mục tiêu	5
1.1 Mục tiêu tổng quát	5
1.2 Mục tiêu cụ thể	5
2 Tổng quan lý thuyết và thư viện sử dụng	6
2.1 Các kỹ thuật băm đặc trưng	6
2.1.1 Hash Table	6
2.1.2 Bloom Filter	8
2.1.3 SimHash	10
2.1.4 MinHash	11
2.2 Thư viện FAISS	12
2.2.1 Giới thiệu chung	12
2.2.2 Cơ chế phân chia không gian (IVF)	12
2.2.3 Nén dữ liệu với Product Quantization (PQ)	13
2.2.4 Tìm kiếm chính xác với Flat Index	13
2.2.5 Tối ưu hóa phần cứng	13
2.3 Mô hình Deep Learning cho trích xuất đặc trưng (ResNet)	13
2.3.1 Vấn đề thoái hóa trong mạng sâu	13
2.3.2 Kiến trúc Residual Network	13
2.3.3 Quy trình trích xuất đặc trưng (Feature Extraction)	14
3 Phương pháp đề xuất	14
3.1 Quy trình xử lý tổng quát	14
3.2 Giai đoạn 1: Lọc trùng lặp tuyệt đối với MD5	15
3.3 Giai đoạn 2: Các phương pháp so khớp nội dung	15
3.3.1 Phương pháp 1: SimHash (Dựa trên đặc trưng cục bộ)	15
3.3.2 Phương pháp 2: MinHash (Dựa trên cấu trúc tập hợp)	15
3.3.3 Phương pháp 3: Deep Learning kết hợp FAISS	16
3.4 Cơ chế chọn ảnh đại diện và Đánh giá	16
3.4.1 Chiến lược chọn ảnh đại diện (Representative Selection)	16
3.4.2 Độ đo đánh giá (Evaluation Metrics)	16
4 Kết quả và đánh giá	17
4.1 Tập dữ liệu (Dataset)	17
4.2 Các tiêu chí đánh giá	17
4.3 Kết quả định tính (Qualitative Results)	17
4.4 Kết quả định lượng (Quantitative Results)	18
4.4.1 Tổng hợp hiệu năng	18
4.4.2 Phân tích số liệu	19
4.5 So sánh và đánh giá	19
4.5.1 SimHash và MinHash (Nhóm thuật toán Băm)	19
4.5.2 ResNet + FAISS (Nhóm Học sâu)	19
4.5.3 Kết luận thực nghiệm	20



5	Kết luận và Hướng phát triển	20
5.1	Tổng kết	20
5.2	Hạn chế	20
5.3	Hướng phát triển	21



List of Figures

1	Các thành phần chính của Hash Table [1]	6
2	Kỹ thuật Seperate Chaining	7
3	Kỹ thuật Open Addressing	8
4	Mình họa thao tác chèn phần tử vào Bloom Filter	9
5	Mình họa thao tác kiểm tra phần tử trong Bloom Filter	9
6	Quy trình tổng quát của thuật toán SimHash	10
7	Kết quả phân cụm thực tế sử dụng ResNet50 và FAISS. Hàng trên là ảnh đại diện được chọn, hàng dưới là các thành viên trong nhóm.	18
8	So sánh thời gian xử lý	19
9	So sánh bộ nhớ sử dụng	19
10	So sánh các chỉ số đánh giá	19

List of Tables

1	Bảng so sánh hiệu năng giữa các phương pháp (Dữ liệu thực nghiệm)	18
---	---	----

1 Mục tiêu

1.1 Mục tiêu tổng quát

Mục tiêu chính của đề tài là xây dựng một hệ thống phần mềm có khả năng tự động phát hiện và loại bỏ các ảnh trùng lặp (duplicate) hoặc gần giống nhau (near-duplicate) trong một tập dữ liệu lớn. Hệ thống hướng tới việc tối ưu hóa không gian lưu trữ và chuẩn hóa dữ liệu bằng cách chọn lọc và giữ lại duy nhất một ảnh đại diện chất lượng nhất cho mỗi nhóm ảnh trùng nội dung.

1.2 Mục tiêu cụ thể

Để hiện thực hóa mục tiêu trên, đề tài tập trung giải quyết các nhiệm vụ cốt lõi sau:

- **Nghiên cứu cơ sở lý thuyết:** Tìm hiểu và phân tích các kỹ thuật băm đặc trưng hiện đại bao gồm Hash Table, Bloom Filter, SimHash, MinHash, cũng như kiến trúc của thư viện tìm kiếm tương đồng FAISS.
- **Ứng dụng Deep Learning:** Sử dụng các mô hình học sâu tiền huấn luyện (như ResNet hoặc EfficientNet) để thực hiện trích xuất đặc trưng, ánh xạ nội dung ảnh từ không gian pixel sang không gian vector.
- **Hiện thực hệ thống:** Cài đặt các thuật toán đã nghiên cứu, ưu tiên sử dụng ngôn ngữ C++ để tối ưu hóa hiệu năng và thực hiện binding sang môi trường Python.
- **Đánh giá thực nghiệm:** So sánh chi tiết về độ chính xác, thời gian thực thi và tài nguyên bộ nhớ giữa các phương pháp tự cài đặt và thư viện FAISS nhằm rút ra ưu nhược điểm của từng giải pháp.

2 Tổng quan lý thuyết và thư viện sử dụng

2.1 Các kỹ thuật băm đặc trưng

2.1.1 Hash Table

1. **Mục tiêu:** Hash table (bảng băm) là một cấu trúc dữ liệu giúp lưu trữ và tra cứu dữ liệu theo *key* một cách nhanh chóng, với độ phức tạp trung bình $O(1)$ cho các thao tác tìm kiếm, chèn và xóa.
2. **Cách hoạt động:** Hash table hoạt động dựa trên khái niệm băm, tức là mỗi *key* sẽ được chuyển đổi thành một *index* trong bảng băm thông qua hàm băm (hash function). Dựa trên giá trị hash, dữ liệu được phân chia vào các *bucket*. Nhờ đó, các thao tác tìm kiếm, chèn và xóa chỉ cần thực hiện trong bucket tương ứng với giá trị hash của key.

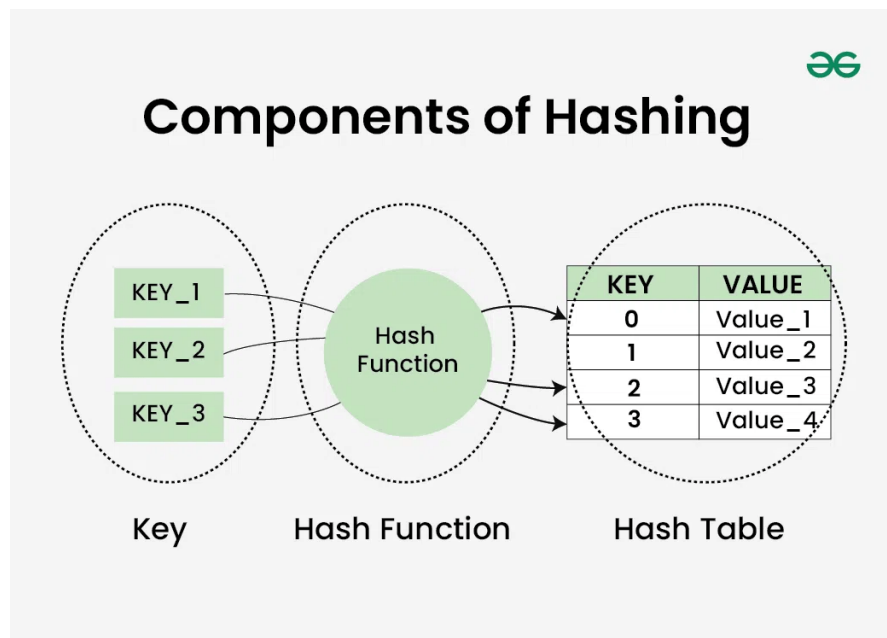


Figure 1: Các thành phần chính của Hash Table [1]

- **Hash function:** là hàm dùng để chuyển đổi *key* thành *index* trong bảng băm:

$$index = f(key, arraySize)$$

Một hàm băm tốt cho ra các giá trị hash phân bố đều, giúp các bucket có kích thước xấp xỉ nhau. Khi số bucket đủ lớn, mỗi bucket chỉ chứa vài phần tử, giúp thao tác tìm kiếm hiệu quả.

- **Hash collision:** Khi có ít nhất hai *key* cho cùng một giá trị hash, ta gọi đó là *hash collision* (va chạm). Việc xử lý va chạm rất quan trọng đối với hiệu năng của bảng băm. Hai phương pháp phổ biến:
 - *Separate chaining:* Mỗi bucket được cài đặt dưới dạng một danh sách liên kết [2].

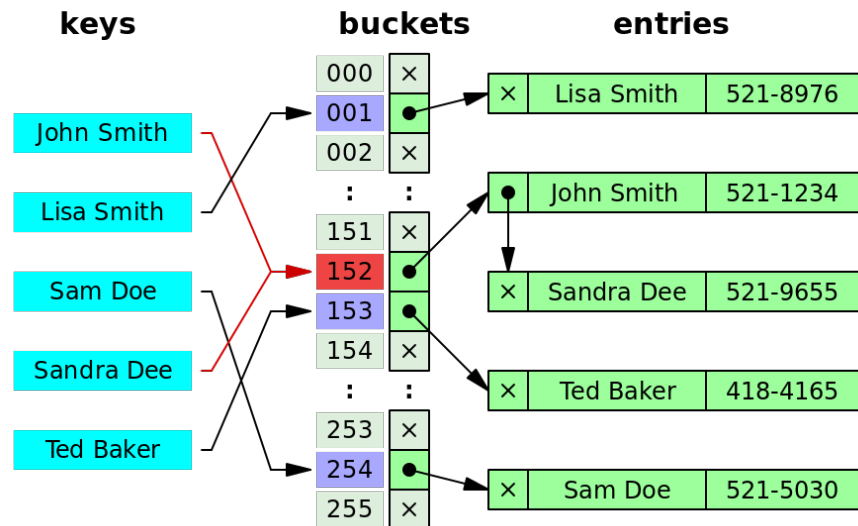


Figure 2: Kỹ thuật Separate Chaining

- *Open addressing*: Khi xảy ra va chạm, phần tử được lưu tại vị trí kế tiếp trong bảng băm. Một số biến thể phổ biến gồm: *linear probing*, *quadratic probing*, và *double hashing* [2].

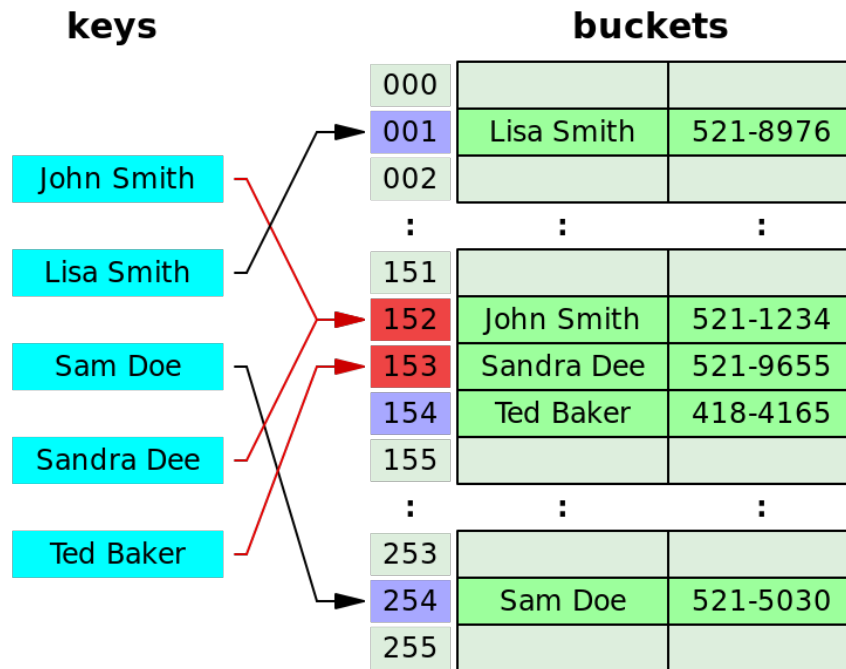


Figure 3: Kỹ thuật Open Addressing

3. **Độ phức tạp:** Gọi n là số phần tử cần lưu và k là số bucket, khi đó:

$$\alpha = \frac{n}{k}$$

được gọi là **load factor**. Khi α nhỏ (xấp xỉ 1) và các giá trị hash phân bố đều, độ phức tạp trung bình cho các thao tác trên bảng băm là $O(1)$. Tuy nhiên, trong trường hợp xấu nhất là khi nhiều giá trị hash trùng nhau thì độ phức tạp có thể tăng lên đến $O(n)$ [2].

2.1.2 Bloom Filter

1. **Mục tiêu:** Bloom Filter là một cấu trúc dữ liệu xác suất, nhanh và tiết kiệm bộ nhớ, dùng để kiểm tra xem một phần tử có thuộc tập hợp hay không. Kết quả trả về là “có thể có” hoặc “chắc chắn không”. Cái giá phải trả cho việc tiết kiệm bộ nhớ là đôi khi cấu trúc có thể trả về kết quả *False Positive*, tức là thông báo phần tử nằm trong tập hợp trong khi thực tế thì không [3].
2. **Cách hoạt động:** Bloom Filter ban đầu là một mảng m bit, tất cả đều có giá trị 0. Cấu trúc này sử dụng k hàm băm độc lập để ánh xạ phần tử đến k chỉ số trong mảng (với $k \ll m$). Hai thao tác chính của Bloom Filter là **chèn (insert)** và **tìm kiếm (lookup)**.

- **Insert:** Để chèn một phần tử x , ta áp dụng k hàm băm để xác định k vị trí trong mảng:

$$index_i = hash_i(x) \% m, \quad 1 \leq i \leq k$$

Sau đó, gán giá trị 1 cho các bit ở các vị trí này.

Ví dụ: Với $k = 3$ và mảng gồm 12 bit, ta có:

$$\text{hash}_1(x_1) = 1, \quad \text{hash}_2(x_1) = 4, \quad \text{hash}_3(x_1) = 8$$

$$\text{hash}_1(x_2) = 4, \quad \text{hash}_2(x_2) = 6, \quad \text{hash}_3(x_2) = 10$$

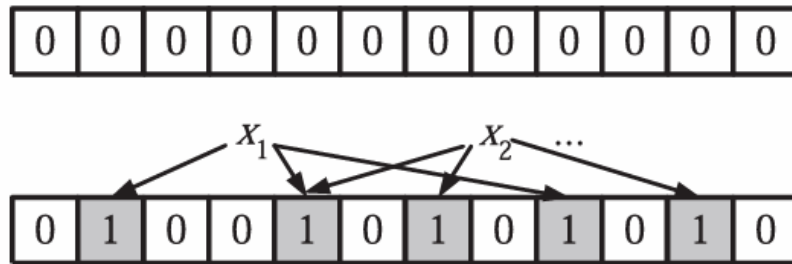


Figure 4: Minh họa thao tác chèn phần tử vào Bloom Filter

- **Lookup:** Để kiểm tra một phần tử y có nằm trong tập hợp hay không, ta tính k hàm băm và kiểm tra các bit tương ứng trong mảng:
 - Nếu tồn tại ít nhất một bit bằng 0 $\Rightarrow$ phần tử *chắc chắn không* nằm trong tập hợp.
 - Nếu tất cả các bit đều bằng 1 $\Rightarrow$ phần tử *có thể* nằm trong tập hợp.

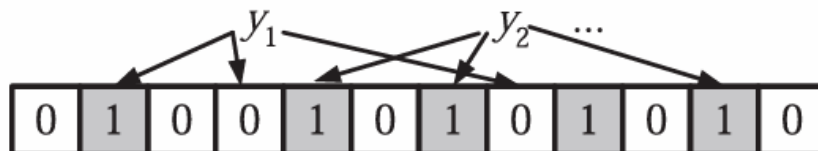


Figure 5: Minh họa thao tác kiểm tra phần tử trong Bloom Filter

Ví dụ: Phần tử y_1 *không* nằm trong tập hợp vì có hai bit 0 tại vị trí 3 và 7. Phần tử y_2 *có thể* nằm trong tập hợp vì tất cả các bit đều bằng 1.

Lưu ý: Bloom Filter **không hỗ trợ thao tác xóa phần tử**, vì việc đặt lại bit về 0 có thể ảnh hưởng đến các phần tử khác.

3. **Độ phức tạp:** Độ phức tạp của các thao tác *insert* và *lookup* phụ thuộc vào số lượng hàm băm k . Mỗi thao tác yêu cầu tính k giá trị hash, nên độ phức tạp là $O(k)$. Với k đủ nhỏ (thường chỉ vài hàm băm), Bloom Filter hoạt động gần như $O(1)$. Tuy nhiên, khi giảm k để tăng tốc độ, xác suất *False Positive* cũng tăng, làm giảm độ chính xác của thuật toán.

2.1.3 SimHash

1. **Mục tiêu:** SimHash là một kỹ thuật băm đặc trưng (feature hashing) được sử dụng để so sánh độ tương tự giữa các đối tượng dữ liệu có kích thước lớn, đặc biệt trong bài toán phát hiện trùng lặp tài liệu hoặc nội dung tương tự. Khác với các bảng băm thông thường (chỉ kiểm tra bằng/khác), SimHash cho phép đo độ tương đồng dựa trên **khoảng cách Hamming** giữa các giá trị băm [4].
2. **Cách hoạt động:** Ý tưởng chính của SimHash là ánh xạ mỗi đối tượng (ví dụ: văn bản, chuỗi đặc trưng, vector) thành một vector nhị phân có độ dài cố định (thường là 64 hoặc 128 bit), sao cho các đối tượng tương tự nhau trong không gian đặc trưng ban đầu sẽ có mã băm tương tự (ít khác bit) [4].

Quy trình tạo SimHash gồm 4 bước:

- **Bước 1: Trích xuất đặc trưng:** Mỗi tài liệu hoặc đối tượng được biểu diễn bởi tập hợp các đặc trưng (feature), ví dụ như từ khóa, token, hoặc vector trọng số TF-IDF.
- **Bước 2: Băm từng đặc trưng:** Mỗi đặc trưng được băm thành một chuỗi nhị phân có độ dài cố định (ví dụ 64 bit).
- **Bước 3: Tính tổng có trọng số:** Với mỗi bit, nếu bit của đặc trưng bằng 1 thì cộng trọng số (thường là trọng số của từ), nếu bằng 0 thì trừ trọng số. Sau khi duyệt hết các đặc trưng, ta thu được một vector số thực $V = [v_1, v_2, \dots, v_n]$.
- **Bước 4: Chuẩn hóa để tạo hash cuối cùng:** Nếu $v_i \geq 0$ thì đặt bit thứ i của SimHash = 1, ngược lại = 0. Kết quả thu được là mã băm đại diện cho toàn bộ đối tượng.

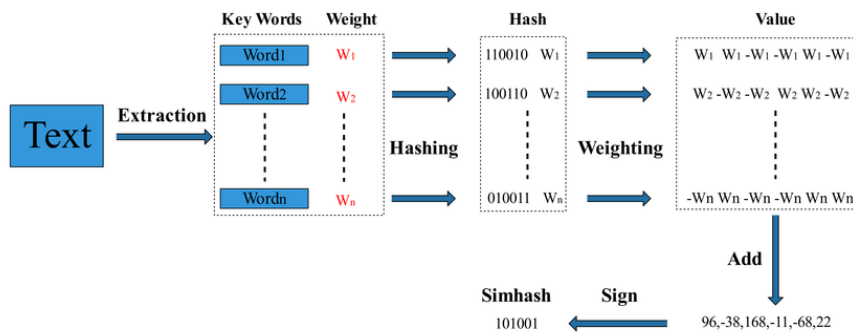


Figure 6: Quy trình tổng quát của thuật toán SimHash

Độ tương tự giữa hai đối tượng được đo bằng **khoảng cách Hamming** giữa hai mã SimHash:

$$\text{similarity}(A, B) = 1 - \frac{\text{HammingDistance}(A, B)}{n}$$

với n là số bit của mã băm (thường 64 hoặc 128).

3. **Ví dụ minh họa:** Giả sử ta có hai văn bản D_1 và D_2 chứa nhiều từ khóa giống nhau. Sau khi tính toán, ta thu được:

$$\text{SimHash}(D_1) = 10110010, \quad \text{SimHash}(D_2) = 10100011$$

Khoảng cách Hamming giữa hai mã này là 2, tức là chúng chỉ khác nhau ở 2 bit, do đó hai văn bản được coi là *rất giống nhau*.

4. **Độ phức tạp:** Giả sử một tài liệu có n đặc trưng, và mỗi đặc trưng được băm thành vector b bit, thì:

- **Tạo SimHash:** $O(n \cdot b)$ (vì cần xử lý từng đặc trưng và cộng dồn cho từng bit)
- **So sánh hai mã SimHash:** $O(b)$ (chỉ cần đếm số bit khác nhau giữa hai mã)

Với b cố định (ví dụ 64), thao tác so sánh có thể xem như $O(1)$, giúp SimHash đặc biệt hiệu quả trong các hệ thống tìm kiếm tài liệu trùng lặp quy mô lớn (như Google hoặc công cụ chống đạo văn).

2.1.4 MinHash

1. **Mục tiêu:** MinHash (viết tắt của *Minimum Hashing*) là một kỹ thuật băm xác suất được sử dụng để ước lượng **độ tương đồng Jaccard** giữa hai tập hợp một cách hiệu quả [5]. Thay vì so sánh trực tiếp toàn bộ các phần tử, MinHash cho phép ta so sánh các giá trị băm đại diện, giúp giảm đáng kể chi phí tính toán và bộ nhớ.

2. **Cách hoạt động:** Ý tưởng của MinHash: Nếu hai tập A và B có nhiều phần tử chung, thì xác suất để hàm băm của phần tử nhỏ nhất (theo giá trị hash) của hai tập trùng nhau sẽ cao. Bằng cách sử dụng nhiều hàm băm khác nhau, ta có thể ước lượng được độ tương đồng Jaccard giữa hai tập hợp mà không cần so sánh từng phần tử.

- **Bước 1: Biểu diễn tập hợp:** Mỗi đối tượng (ví dụ: tài liệu, trang web, đoạn văn) được biểu diễn bằng một tập hợp các đặc trưng, chẳng hạn như tập từ hoặc n-gram.
- **Bước 2: Tạo nhiều hàm băm:** Giả sử ta có k hàm băm $h_1, h_2, \dots, h_k$ độc lập.
- **Bước 3: Tính chữ ký MinHash:** Với mỗi hàm băm h_i , ta tìm giá trị nhỏ nhất khi áp dụng hàm đó cho tất cả phần tử trong tập:

$$M_i(A) = \min_{x \in A} h_i(x)$$

Chữ ký (signature) của tập A được định nghĩa là:

$$\text{Sig}(A) = [M_1(A), M_2(A), \dots, M_k(A)]$$

- **Bước 4: Tính độ tương đồng:** Độ tương đồng Jaccard giữa hai tập A và B được xấp xỉ bởi tỷ lệ số phần tử trùng nhau trong chữ ký:

$$\text{sim}(A, B) = \frac{|\{i : M_i(A) = M_i(B)\}|}{k}$$

Trực quan: Nếu ta coi mỗi hàng là một phần tử và mỗi cột là một tập, MinHash lấy “phần tử đầu tiên” (theo thứ tự băm) xuất hiện trong mỗi cột. Hai cột (tập hợp) có càng nhiều hàng trùng nhau $\rightarrow$ tương đồng càng cao.

3. **Ví dụ minh họa:** Giả sử ta có hai tập:

$$A = \{1, 3, 4, 5\}, \quad B = \{2, 3, 5, 6\}$$

và ba hàm băm giả định:

$$h_1(x) = (2x + 1) \bmod 7, \quad h_2(x) = (3x + 2) \bmod 7, \quad h_3(x) = (5x + 2) \bmod 7$$

Khi đó, giá trị nhỏ nhất cho mỗi hàm băm trên từng tập là:

$$\text{Sig}(A) = [1, 0, 3], \quad \text{Sig}(B) = [1, 0, 2]$$

Hai chữ ký trùng nhau ở 2 trên 3 vị trí, nên ta có:

$$\text{sim}(A, B) \approx \frac{2}{3} = 0.67$$

→ Hai tập tương đồng khoảng 67%.

4. **Độ phức tạp:** Giả sử mỗi tập có n phần tử và sử dụng k hàm băm:

- **Tạo chữ ký MinHash:** $O(n \cdot k)$ (vì cần tính giá trị nhỏ nhất cho mỗi hàm băm)
- **So sánh hai tập hợp:** $O(k)$ (so sánh trực tiếp hai vector chữ ký)

2.2 Thư viện FAISS

2.2.1 Giới thiệu chung

FAISS (Facebook AI Similarity Search) là thư viện mã nguồn mở được phát triển bởi Meta AI, đóng vai trò là chuẩn mực công nghiệp hiện nay cho bài toán tìm kiếm tương đồng trên tập dữ liệu quy mô lớn [6]. Vấn đề cốt lõi mà FAISS giải quyết là "lời nguyền của số chiều" (curse of dimensionality) khi tìm kiếm trong không gian vector mật độ cao.

Nếu sử dụng phương pháp tìm kiếm vét cạn (Brute-force hay Flat Index), độ phức tạp tính toán là $O(N \cdot D)$, với N là số lượng ảnh và D là số chiều vector. Khi N đạt ngưỡng hàng triệu, độ trễ sẽ vượt quá mức chấp nhận được của các ứng dụng thời gian thực. FAISS khắc phục điều này bằng cách xây dựng các chỉ mục (Index) dựa trên giải thuật Tìm kiếm Cận láng giềng Gần đúng (Approximate Nearest Neighbor - ANN), chấp nhận đánh đổi một lượng nhỏ độ chính xác (Recall) để đạt được tốc độ truy vấn cực nhanh.

2.2.2 Cơ chế phân chia không gian (IVF)

Để tránh việc so sánh vector truy vấn với toàn bộ dữ liệu, FAISS sử dụng kỹ thuật Inverted File Index (IVF). Không gian vector được phân hoạch thành các vùng Voronoi (Voronoi cells) thông qua thuật toán phân cụm k-means.

- **Quá trình huấn luyện (Train):** Hệ thống học ra k tâm cụm (centroids) đại diện cho phân bố dữ liệu.
- **Quá trình thêm (Add):** Mỗi vector ảnh v_i sẽ được gán vào danh sách (inverted list) của tâm cụm gần nhất.
- **Quá trình tìm kiếm (Search):** Khi có một truy vấn q , FAISS xác định tâm cụm gần nhất với q và chỉ thực hiện quét tuyến tính các vector nằm trong cụm đó (và một số cụm lân cận được quy định bởi tham số `nprobe`).

Phương pháp này giúp giảm không gian tìm kiếm từ toàn bộ N phần tử xuống chỉ còn một tập con nhỏ, tăng tốc độ truy vấn đáng kể.

2.2.3 Nén dữ liệu với Product Quantization (PQ)

Một thách thức lớn khác là chi phí bộ nhớ. Một tập dữ liệu 1 triệu ảnh sử dụng ResNet-50 (vector 2048 chiều, float32) sẽ tiêu tốn khoảng 8GB RAM. FAISS giải quyết vấn đề này bằng kỹ thuật Lượng tử hóa tích (Product Quantization - PQ).

PQ chia vector gốc có số chiều D thành M vector con (sub-vectors). Mỗi không gian con này được lượng tử hóa độc lập bằng một bộ mã (codebook) riêng. Thay vì lưu trữ giá trị thực (float), hệ thống chỉ cần lưu trữ chỉ số (index) của tâm cụm trong codebook tương ứng. Kỹ thuật này có thể nén kích thước vector xuống hàng chục lần (ví dụ từ 2048 floats xuống 64 bytes) giúp lưu trữ hàng tỷ vector trên bộ nhớ RAM giới hạn.

2.2.4 Tìm kiếm chính xác với Flat Index

Mặc dù các kỹ thuật IVF và PQ rất mạnh mẽ cho dữ liệu quy mô lớn (xấp xỉ), FAISS cũng cung cấp các chỉ mục cho tìm kiếm chính xác tuyệt đối (Exact Search), tiêu biểu là **IndexFlatL2** hoặc **IndexFlatIP**.

- **Cơ chế:** Lưu trữ nguyên vẹn các vector trong bộ nhớ mà không nén hay phân cụm. Khi truy vấn, thuật toán thực hiện so sánh vét cạn (Brute-force) giữa vector truy vấn với toàn bộ N vector trong cơ sở dữ liệu.
- **Ưu điểm:** Đảm bảo độ chính xác (Recall) gần như là 100% vì không có sai số do lượng tử hóa.
- **Ứng dụng:** Phù hợp cho các tập dữ liệu có kích thước trung bình (dưới 1 triệu vector) hoặc khi yêu cầu độ chính xác là ưu tiên hàng đầu so với tốc độ. Trong ngữ cảnh của đề tài này, với tập dữ liệu ảnh thử nghiệm chưa đạt ngưỡng "Big Data", việc sử dụng **IndexFlat** là lựa chọn tối ưu để đánh giá chất lượng trích xuất đặc trưng.

2.2.5 Tối ưu hóa phần cứng

FAISS được tối ưu hóa sâu ở mức phần cứng. Trên CPU, thư viện tận dụng các chỉ lệnh vector hóa (SIMD) như AVX2/AVX-512 và xử lý đa luồng (OpenMP). Trên GPU, FAISS sử dụng nhân CUDA tùy chỉnh để song song hóa quá trình tính toán khoảng cách và sắp xếp (k-selection), cho phép thực hiện hàng triệu truy vấn mỗi giây trên các dòng card đồ họa NVIDIA phổ thông.

2.3 Mô hình Deep Learning cho trích xuất đặc trưng (ResNet)

2.3.1 Vấn đề thoái hóa trong mạng sâu

Trong thị giác máy tính, độ sâu của mạng nơ-ron đóng vai trò quyết định đến khả năng trích xuất các đặc trưng trừu tượng. Tuy nhiên, các mạng CNN truyền thống (như VGG) gặp phải vấn đề biến mất đạo hàm (vanishing gradient) khi độ sâu tăng lên, dẫn đến việc mạng bão hòa và suy giảm độ chính xác huấn luyện.

2.3.2 Kiến trúc Residual Network

ResNet [7] giải quyết vấn đề này bằng cơ chế "Học thặng dư" (Residual Learning). Đơn vị cấu trúc cơ bản của ResNet là khối thặng dư (Residual Block) với công thức:

$$y = \mathcal{F}(x, \{W_i\}) + x \quad (1)$$

Trong đó:

- x và y là vector đầu vào và đầu ra của các lớp.
- $\mathcal{F}(x, \{W_i\})$ là hàm ánh xạ thẳng dư cần học.
- toán tử $+x$ biểu thị kết nối tắt (skip connection).

Cấu trúc này cho phép luồng gradient truyền trực tiếp qua kết nối tắt trong quá trình lan truyền ngược (backpropagation), đảm bảo khả năng huấn luyện cho các mạng rất sâu (lên tới 152 lớp hoặc hơn).

2.3.3 Quy trình trích xuất đặc trưng (Feature Extraction)

Để phục vụ bài toán tìm kiếm ảnh tương đồng, đề tài sử dụng kiến trúc ResNet-50 đã được huấn luyện trước (pre-trained) trên tập dữ liệu ImageNet. Quy trình trích xuất đặc trưng được thực hiện như sau:

1. **Tiền xử lý:** Ảnh đầu vào được thay đổi kích thước về 224×224 pixel và chuẩn hóa theo phân phối của ImageNet (mean và std).
2. **Lan truyền thuận (Forward Pass):** Ảnh đi qua các tầng tích chập (Convolutional layers) để trích xuất các đặc trưng từ mức thấp (cạnh, góc) đến mức cao (hình dạng, vật thể).
3. **Lớp Global Average Pooling (GAP):** Tại lớp cuối cùng trước khi phân loại, tensor đặc trưng có kích thước (7, 7, 2048). Lớp GAP sẽ tính giá trị trung bình trên miền không gian 7×7 , nén tensor này thành một vector duy nhất có kích thước 2048 chiều.
4. **Embedding Vector:** Lớp Fully Connected (FC) cuối cùng dùng để phân lớp được loại bỏ. Vector 2048 chiều thu được từ lớp GAP chính là đại diện số học (embedding) cho nội dung bức ảnh, được sử dụng làm đầu vào cho các thuật toán băm và tìm kiếm FAISS.

3 Phương pháp đề xuất

3.1 Quy trình xử lý tổng quát

Hệ thống được hiện thực hóa dựa trên quy trình đường ống (pipeline) gồm 4 giai đoạn chính để xử lý tập dữ liệu ảnh đầu vào. Quy trình này được thiết kế để loại bỏ các ảnh trùng lặp tuyệt đối trước khi áp dụng các thuật toán so khớp tốn kém hơn.

1. **Tiền xử lý (Preprocessing):** Chuẩn hóa kích thước và màu sắc ảnh.
2. **Lọc trùng lặp tuyệt đối (Exact Deduplication):** Sử dụng mã băm MD5.
3. **Trích xuất và Băm đặc trưng (Hashing/Embedding):** Áp dụng song song ba phương pháp để so sánh: SimHash, MinHash và Deep Learning (FAISS).
4. **Gom nhóm và Đánh giá:** Phân cụm các ảnh tương đồng và chọn ảnh đại diện.

3.2 Giai đoạn 1: Lọc trùng lặp tuyệt đối với MD5

Trước khi đi vào các phân tích nội dung phức tạp, hệ thống thực hiện một bước lọc sơ cấp để loại bỏ các tệp ảnh hoàn toàn giống nhau về mặt nhị phân (bit-by-bit).

- **Thuật toán:** MD5 (Message-Digest Algorithm 5).
- **Hiện thực:** Đọc nội dung file dưới dạng binary và tính chuỗi mã băm 128-bit.
- **Cơ chế lọc:** Các ảnh có cùng mã MD5 sẽ được gom vào cùng một nhóm. Chỉ một ảnh duy nhất được giữ lại để đưa vào các giai đoạn xử lý tiếp theo, giúp giảm tải khối lượng tính toán.

3.3 Giai đoạn 2: Các phương pháp so khớp nội dung

Đề tài thực nghiệm và so sánh ba phương pháp tiếp cận khác nhau để phát hiện ảnh gần giống (near-duplicate).

3.3.1 Phương pháp 1: SimHash (Dựa trên đặc trưng cục bộ)

Phương pháp này tập trung vào sự tương đồng về mặt nhận thức (perceptual similarity) thông qua phân bố độ sáng.

- **Tiền xử lý:** Ảnh được chuyển sang ảnh xám (Grayscale) và thay đổi kích thước về 64×64 pixel để giảm nhiễu chi tiết.
- **Trích xuất đặc trưng:** Ảnh được chia thành các khối (block) kích thước 8×8 . Giá trị trung bình pixel của mỗi khối được tính toán để tạo thành vector đặc trưng.
- **Fingerprinting:** Sử dụng thư viện `simhash` để tạo chữ ký bit.
- **So khớp:** Sử dụng khoảng cách Hamming. Hai ảnh được coi là trùng lặp nếu khoảng cách Hamming ≤ 16 (ngưỡng thực nghiệm).

3.3.2 Phương pháp 2: MinHash (Dựa trên cấu trúc tập hợp)

MinHash được sử dụng để ước lượng độ tương đồng Jaccard giữa các tập hợp đặc trưng hình ảnh.

- **Xây dựng tập đặc trưng (Shingling):**
 - Ảnh được resize về 64×64 .
 - Duyệt qua các block 8×8 (stride = 8).
 - Mức sáng trung bình của block được lượng tử hóa thành 16 mức (levels).
 - Mỗi block được mã hóa thành một chuỗi string: `"b_{row}_{col}_{level}"`. Tập hợp các chuỗi này đại diện cho cấu trúc của ảnh.
- **Locality Sensitive Hashing (LSH):** Sử dụng thư viện `datasketch` với số lượng hàm hoán vị (`num_perm`) là 128.
- **Truy vấn:** Sử dụng chỉ mục LSH để tìm các ảnh có độ tương đồng Jaccard ≥ 0.6 .

3.3.3 Phương pháp 3: Deep Learning kết hợp FAISS

Đây là phương pháp tiếp cận hiện đại sử dụng mạng nơ-ron tích chập để trích xuất ngữ nghĩa.

- **Mô hình Backbone:** Sử dụng ResNet50 đã được huấn luyện trước (pretrained) trên tập ImageNet. Lớp phân loại cuối cùng bị loại bỏ để lấy vector embedding (kích thước 2048).
- **Chuẩn hóa dữ liệu:**
 - Resize ảnh về 224×224 .
 - Chuyển đổi sang Tensor và chuẩn hóa theo phân phối ImageNet (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]).
- **Indexing với FAISS:**
 - Các vector đặc trưng được chuẩn hóa theo chuẩn L_2 (Euclidean norm).
 - Sử dụng chỉ mục IndexFlatIP (Inner Product) của FAISS. Do vector đã được chuẩn hóa, tích vô hướng tương đương với độ tương đồng Cosine.
- **Gom nhóm:** Thực hiện tìm kiếm k -Nearest Neighbors (với $k = 10$). Các ảnh có độ tương đồng Cosine ≥ 0.6 được gom vào cùng một nhóm.

3.4 Cơ chế chọn ảnh đại diện và Đánh giá

3.4.1 Chiến lược chọn ảnh đại diện (Representative Selection)

Với mỗi nhóm ảnh trùng lặp được phát hiện, hệ thống cần giữ lại một ảnh tốt nhất. Thuật toán lựa chọn dựa trên độ phân giải:

$$Score(I) = Width(I) \times Height(I) \quad (2)$$

Ảnh có điểm $Score$ (diện tích) lớn nhất trong nhóm sẽ được chọn làm ảnh đại diện.

3.4.2 Độ đo đánh giá (Evaluation Metrics)

Để đánh giá hiệu quả của từng phương pháp, hệ thống sử dụng tập dữ liệu có nhãn (ground truth) dựa trên quy tắc đặt tên file (ví dụ: các file có cùng prefix `group_id` thuộc về một nhóm). Các chỉ số được tính toán bao gồm:

- **Precision (Độ chính xác):** Tỷ lệ các cặp ảnh được dự đoán là trùng lặp thực sự là trùng lặp.
- **Recall (Độ phủ):** Tỷ lệ các cặp ảnh trùng lặp thực tế được hệ thống phát hiện.
- **F1-Score:** Trung bình điều hòa giữa Precision và Recall.
- **Tài nguyên:** Thời gian thực thi (Processing Time) và dung lượng bộ nhớ tiêu thụ (Memory Usage).

4 Kết quả và đánh giá

4.1 Tập dữ liệu (Dataset)

Quá trình thực nghiệm được tiến hành trên tập dữ liệu mẫu bao gồm $N = 513$ ảnh (ban đầu). Các ảnh được sinh ra có chủ đích để tạo thành các nhóm trùng lặp.

- **Quy tắc nhãn (Ground Truth):** Nhãn thực tế được xác định dựa trên quy tắc đặt tên tệp tin. Các ảnh thuộc cùng một nhóm trùng lặp sẽ có cùng tiền tố (prefix) trong tên file (ví dụ: `f117_1.jpg` và `f117_2.jpg` thuộc cùng nhóm `f117`).
- **Môi trường thực thi:** Hệ thống được triển khai và đánh giá trên môi trường Google Colab.

4.2 Các tiêu chí đánh giá

Hiệu năng của hệ thống được đánh giá dựa trên 3 khía cạnh chính:

1. **Độ chính xác phân cụm:** Sử dụng các chỉ số Precision, Recall và F1-Score dựa trên cặp ảnh (pairwise metrics).

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F1 = 2 \cdot \frac{P \cdot R}{P + R} \quad (3)$$

Trong đó TP là số cặp ảnh trùng lặp được phát hiện đúng, FP là số cặp báo trùng sai, FN là số cặp trùng lặp bị bỏ sót.

2. **Thời gian xử lý (Latency):** Tổng thời gian từ lúc nạp ảnh đến khi xuất ra danh sách nhóm trùng lặp.
3. **Tài nguyên (Memory):** Lượng RAM tiêu thụ trong quá trình thực thi (đo bằng thư viện `psutil`).

4.3 Kết quả định tính (Qualitative Results)

Để làm rõ khả năng nhận diện của hệ thống Deep Learning (phương pháp có F1-Score cao nhất), Hình 7 trực quan hóa một số cụm ảnh tiêu biểu được phát hiện.



Figure 7: Kết quả phân cụm thực tế sử dụng ResNet50 và FAISS. Hàng trên là ảnh đại diện được chọn, hàng dưới là các thành viên trong nhóm.

Quan sát kết quả định tính cho thấy:

- **Kháng biến đổi hình học:** Tại cột thứ 3 (hoa loa kèn) và cột thứ 5 (hoa dại), hệ thống gom nhóm thành công các ảnh bị xoay góc (rotation) hoặc lật (flip). Điều này chứng tỏ vector đặc trưng từ ResNet có tính bất biến cao với các phép biến đổi affine.
- **Kháng nhiễu màu sắc:** Tại cột thứ 4 (phong cảnh), dù ảnh bị thay đổi tông màu (filter) và độ sáng, hệ thống vẫn nhận diện chính xác.
- **Lựa chọn ảnh đại diện:** Thuật toán tự động chọn được ảnh có độ phân giải tốt nhất và bố cục rõ ràng nhất làm đại diện (hàng trên cùng), đáp ứng mục tiêu tối ưu hóa lưu trữ.

4.4 Kết quả định lượng (Quantitative Results)

Mục này trình bày các số liệu thống kê thu được sau khi chạy thực nghiệm trên toàn bộ tập dữ liệu.

4.4.1 Tổng hợp hiệu năng

Bảng 1 tóm tắt kết quả so sánh giữa ba phương pháp tiếp cận: MD5 (lọc trùng lặp chính xác), các thuật toán Băm (SimHash, MinHash) và Học sâu (FAISS + ResNet50).

Table 1: Bảng so sánh hiệu năng giữa các phương pháp (Dữ liệu thực nghiệm)

Phương pháp	Time (s)	Mem (MB)	Precision	Recall	F1-Score
MD5 (Baseline)	5.20	≈ 0.1	1.00	N/A	N/A
SimHash (C++)	27.27	0.37	0.93	0.24	0.38
MinHash (LSH)	32.80	6.23	0.84	0.23	0.37
FAISS (ResNet)	215.92	527.62	0.83	0.78	0.80

4.4.2 Phân tích số liệu

- **Về tốc độ:** Các thuật toán băm thể hiện ưu thế vượt trội. SimHash chỉ mất khoảng 27 giây để xử lý, nhanh gấp gần 8 lần so với phương pháp Deep Learning (215 giây). Điều này là do chi phí tính toán forward-pass qua mạng ResNet-50 rất lớn.
- **Về độ chính xác (F1-Score):** FAISS đạt F1-Score cao nhất (0.80), cân bằng tốt giữa Precision và Recall. Trong khi đó, SimHash và MinHash có chỉ số Recall rất thấp (0.24), nghĩa là chúng bỏ sót khoảng 76% các cặp ảnh trùng lặp (False Negatives).

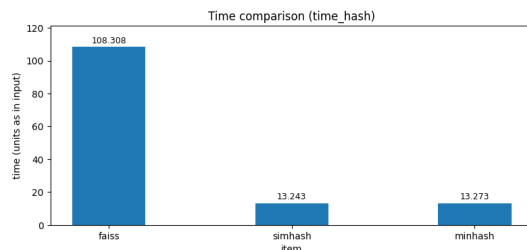


Figure 8: So sánh thời gian xử lý

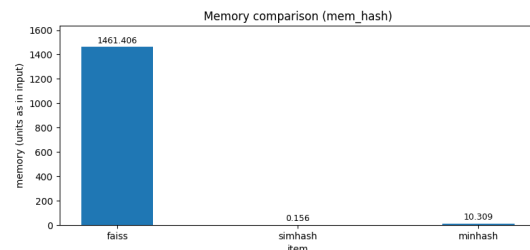


Figure 9: So sánh bộ nhớ sử dụng

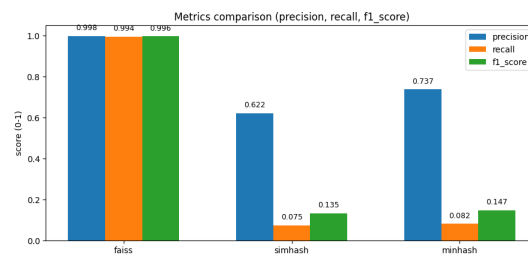


Figure 10: So sánh các chỉ số đánh giá

4.5 So sánh và đánh giá

Từ các kết quả định lượng và định tính trên, ta có thể rút ra các nhận định so sánh chi tiết giữa ba phương pháp:

4.5.1 SimHash và MinHash (Nhóm thuật toán Băm)

- **Ưu điểm:** Tốc độ cực nhanh và chi phí bộ nhớ rất thấp (chỉ tốn vài MB). Phù hợp cho bài toán lọc sơ cấp trên dữ liệu quy mô cực lớn (Web-scale).
- **Nhược điểm:** Rất nhạy cảm với các biến đổi hình học. Chỉ cần ảnh bị xoay nhẹ hoặc dịch chuyển, cấu trúc bit của mã băm thay đổi lớn (do tính chất của thuật toán dựa trên phân bố pixel/tần suất cục bộ), dẫn đến khoảng cách Hamming tăng cao và **Recall thấp**.

4.5.2 ResNet + FAISS (Nhóm Học sâu)

- **Ưu điểm:** Độ chính xác vượt trội nhờ khả năng trích xuất đặc trưng ngữ nghĩa (Semantic Features). Mạng CNN học được các đặc điểm trừu tượng (hình dáng, vật thể) thay vì chỉ so sánh pixel, do đó kháng tốt với việc cắt xén, xoay, đổi màu.

- **Nhược điểm:** Yêu cầu tài nguyên phần cứng cao (GPU) và tốn bộ nhớ lưu trữ vector (527MB cho tập dữ liệu nhỏ).

4.5.3 Kết luận thực nghiệm

Không có phương pháp nào là hoàn hảo tuyệt đối. Một hệ thống thực tế tối ưu nên áp dụng mô hình **Phân tầng (Cascading)**:

1. Dùng **MD5** để loại bỏ nhanh trùng lặp tuyệt đối (Chi phí ≈ 0).
2. Dùng **SimHash** để lọc các ảnh gần giống về cấu trúc cơ bản (Chi phí thấp).
3. Dùng **FAISS** cho tập dữ liệu còn lại để phát hiện các ca khó, đảm bảo chất lượng cao nhất cho người dùng cuối.

5 Kết luận và Hướng phát triển

5.1 Tổng kết

Đề tài đã xây dựng và đánh giá thành công hệ thống phát hiện ảnh trùng lặp và gần giống nhau trên tập dữ liệu thực tế. Dựa trên các kết quả thực nghiệm, nhóm nghiên cứu rút ra các kết luận sau:

- **Hiệu quả của Deep Learning:** Phương pháp sử dụng Vector Embedding (ResNet50 kết hợp FAISS) chứng tỏ ưu thế vượt trội trong việc phát hiện các ảnh tương đồng về mặt ngữ nghĩa (semantic similarity) với F1-Score đạt **80%**. Hệ thống có khả năng kháng lại các biến đổi phức tạp như xoay, cắt xén và thay đổi ánh sáng.
- **Sự đánh đổi (Trade-off):** Các thuật toán băm truyền thống (SimHash, MinHash) có tốc độ xử lý cực nhanh (nhanh hơn 8 lần so với Deep Learning) và chi phí bộ nhớ thấp, phù hợp cho bài toán lọc thô. Tuy nhiên, độ chính xác của chúng giảm mạnh khi ảnh bị biến đổi cấu trúc.
- **Chiến lược tối ưu:** Đề xuất mô hình kết hợp đa tầng: sử dụng MD5 để loại bỏ trùng lặp tuyệt đối, SimHash để lọc sơ bộ và FAISS để tinh chỉnh kết quả cuối cùng.

5.2 Hạn chế

Bên cạnh những kết quả đạt được, hệ thống vẫn còn một số hạn chế:

- **Chi phí tài nguyên:** Mô hình ResNet-50 yêu cầu dung lượng RAM lớn và GPU để đạt tốc độ thời gian thực, gây khó khăn khi triển khai trên các thiết bị biên (edge devices) hoặc máy chủ cấu hình thấp.
- **Ngưỡng cố định:** Các ngưỡng khoảng cách (Threshold) hiện tại được thiết lập dựa trên thực nghiệm với tập dữ liệu cụ thể, chưa có cơ chế tự động thích nghi (adaptive) với các miền dữ liệu khác nhau.

5.3 Hướng phát triển

Dựa trên nền tảng hệ thống đã hoàn thiện, các hướng nghiên cứu nâng cao tiếp theo bao gồm:

1. **Học đo lường khoảng cách (Metric Learning):** Hiện tại hệ thống đang sử dụng mô hình pretrained trên ImageNet (phân loại vật thể). Trong tương lai, có thể fine-tune lại mạng sử dụng *Siamese Network* với *Triplet Loss* hoặc *ArcFace*. Điều này giúp mạng học cách "kéo" các vector ảnh trùng lặp lại gần nhau hơn và "đẩy" các ảnh khác nhau ra xa hơn trong không gian nhúng, thay vì chỉ dựa vào đặc trưng phân loại chung chung.
2. **Mở rộng sang Video:** Áp dụng kỹ thuật trích xuất đặc trưng theo khung hình (Keyframe Extraction) để phát hiện các đoạn video trùng lặp hoặc vi phạm bản quyền, mở rộng phạm vi ứng dụng của hệ thống.
3. **Triển khai hệ thống phân tán:** Với tập dữ liệu quy mô hàng triệu ảnh, việc xử lý trên một máy đơn lẻ là không khả thi. Hướng phát triển tiếp theo là tích hợp *Apache Spark* hoặc *Dask* để song song hóa quá trình trích xuất đặc trưng và sử dụng *FAISS GPU Cluster* để tìm kiếm đa luồng.

References

- [1] GeeksforGeeks, “Hash table data structure.” <https://www.geeksforgeeks.org/dsa/hash-table-data-structure/>, 2024. Truy cập ngày: 04/10/2025.
- [2] VNOI Wiki, “Hash table (bảng băm).” <https://wiki.vnoi.info/algo/data-structures/hash-table/>, 2025. Truy cập ngày: 04/10/2025.
- [3] A. Broder and M. Mitzenmacher, “Network applications of bloom filters: A survey,” *Internet mathematics*, vol. 1, no. 4, pp. 485–509, 2004.
- [4] Maneshwar, “What is SimHash?.” <https://dev.to/lovestaco/what-is-simhash-58m5>, 2023. Dev.to. Truy cập ngày: 03/12/2025.
- [5] J. S. Plank, “Cs494 lecture notes - minhash.” <https://web.eecs.utk.edu/~jplank/plank/classes/cs494/494/notes/Min-Hash/index.html>, 2024. University of Tennessee. Truy cập ngày: 03/12/2025.
- [6] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.
- [7] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, IEEE, 2016.